

מבנה שפות תוכנה – 3

רקורסיות

תכנות פונקציונלי נקרא "טהור" אם אין בו שינוי מצב, כלומר אין בו משתנים או אובייקטים שמשנים את מצבם הפנימי. כל עיבוד מידע נעשה על ידי יצירת ערכים חדשים כנדרש.

בתרגיל זה נדרש לפתור כל סעיף ב 2 דרכים ע"פ בגישה הפונקציונלית – ללא לולאות ומשתנים כלל, באמצעות:

- רקורסיה רגילה
 - רקורסיה זנבית (פונקציה רקורסיבית המתוכננת כך שתאפשר ריצה בסיבוכיות מקום נמוכה יותר מאשר סיבוכיות מקום פרופורציונלית לעומק מחסנית הרקורסיה)
1. צרו באמצעות רקורסיה tuple של המספרים 1-1000 כלומר (1,2,3,...,1000)
 2. כתבו פונקציה רקורסיבית המחשבת סכום איברי מערך, הריצו אותה על tuple שיצרתם (בסעיף הקודם)
 3. כתבו פונקציה הפותרת את בעיית LCM – מכפיל משותף מינימאלי $LCM(6,4)=12$
 4. כתבו פונקציה רקורסיבית בוליאנית is_palindrome_number המקבלת מספר שלם ובודקת אם הוא פלינדרום למשל 123454321
 5. מחרוזת נקראת "פלינדרום אלפאנומרי" אם היא נקראת אותו דבר מהתחלה לסוף ומהסוף להתחלה, תוך התעלמות מרווחים, סימני פיסוק והבדלי אותיות גדולות/קטנות (Case-Insensitive).
- למשל: המחרוזת "A man, a plan, a canal: Panama" היא פלינדרום אלפאנומרי.
- כתבו פונקציה טהורה בשם is_palindrome_alphanumeric המקבלת מחרוזת ומחזירה True אם היא פלינדרום אלפאנומרי ו-False אחרת.
- על הפונקציה לבצע את ניקוי התווים הבלתי-רלוונטיים וההמרה לאותיות קטנות בעצמה ובאופן פונקציונלי.

סעיף אתגר (עברית ואותיות סופיות):

יש לתמוך גם במחרוזת בעברית! בבדיקה בעברית, יש להתייחס לאותיות סופיות כאותיות רגילות לכל דבר (למשל: 'ך' שקולה ל-'כ', 'ם' שקולה ל-'מ', 'ן' שקולה ל-'נ', 'ף' שקולה ל-'פ', ו-'ץ' שקולה ל-'צ').

לדוגמה: המחרוזת "נטול גלוטן" היא פלינדרום תקין כיוון ש-'נ' סופית שקולה ל-'נ'.

בנוסף, כתבו סקריפט ראשי המקבל מחרוזת מהמשתמש, מפעיל את הפונקציה `is_palindrome` ומדפיס את התוצאה (`True` או `False`).

אם הקלט ריק (או מכיל רק רווחים), על הסקריפט להדפיס `"invalid input"`.

דוגמאות להרצה:

enter text:

A man, a plan, a canal: Panama

True

enter text:

נטול גלוטן

True

enter text:

Hello World

False

enter text:

invalid input

6. כתבו פונקציה רקורסיבית `sortedzip` שמקבלת רשימה של רשימות, ממיינת אותן בסדר עולה (בלי למיין במקום, תוך שימוש של הפונקציה `sorted`, ולא המתודה `sort`!!) ומחזירה את ה-`zip` שלהן. למשל:

```
>>> list(sortedzip([[3,1,2],[5,6,4],['a','b','c']])) [(1, 4, 'a'), (2, 5, 'b'), (3, 6, 'c')]
```

7. דחיסת RLE היא טכניקה פשוטה שבה רצפים של תווים זהים עוקבים מוחלפים בתו היחיד ולצידו מספר החזרות שלו.

למשל: המחרוזת `"AAABBCDDDD"` תידחס ל-`"A3B2C1D4"`.

כתבו פונקציה טהורה ורקורסיבית בשם `encode_rle` המקבלת מחרוזת ומחזירה מחרוזת מוקדדת לפי כלל זה.

על הפונקציה לשמור על עקרונות התכנות הפונקציונלי: אי-שתנות (Immutability), הימנעות ממשתנים גלובליים, והימנעות מלולאות (for/while) או ספריות איטרטיביות.

בנוסף, כתבו סקריפט ראשי המקבל מחרוזת מהמשתמש, מפעיל את הפונקציה `encode_rle` ומדפיס את התוצאה.

אם הקלט ריק (או מכיל רק רווחים), על הסקריפט להדפיס `"invalid input"`.

דוגמאות להרצה:

enter text:

AAABBCDDDD

A3B2C1D4

enter text:

a

a1

enter text:

invalid input

Lazy Evaluation, Generators

בכל התרגילים, אם לא נאמר אחרת, יש לכתוב בצורה הפונקציונלית הטובה ביותר.

1. לפניך שתי משימות, את כל אחת מהמשימות חובה לבצע בשתי שיטות:

- ללא שימוש ב lazy evaluation
- תוך שימוש ב lazy evaluation

א צרו מערך המכיל את המספרים 0-10000

כתבו את הקוד בפונקציה עצמאית

מדדו זמן הביצוע של הפונקציה ואת גודל האובייקט בזיכרון

ב צרו מהמערך הקודם מערך המכיל רק את 5000 האברים הראשונים

מדדו את זמן הביצוע וגודל הזיכרון

בדקו את `type` של המערך המלא וודאו שיש למערך החדש את אותו `type`

2. כתבו גנרטור לחישוב מספרים ראשוניים, בכל פניה לגנרטור ע"י next הוא יחזיר את המספר הראשוני הבא.

3. ממשו את שאלה 4 מהתרגיל הקודם (חישוב טור טיילור) ע"י גנרטורים כך שבכל פנייה לגנרטור יתווסף לחישוב האיבר הבא בטור
דוג לפלט עבור הקלט 2 כלומר e^2 :

1.0
3.0
5.0
6.333333333333333
7.0
7.266666666666667
7.355555555555555
7.3809523809523805